Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-214БВ-25

Студент: Степанова А.М.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 02.11.26

Москва, 2026

Постановка задачи

Вариант 19.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. Основной процесс должен создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через отображаемые файлы (memory-mapped files). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Задание: переделать лабораторную работу №1 с использованием shared memory и memory mapping. Для синхронизации чтения и записи в общей памяти используется семафор. Общая память должна быть именованным объектом в оперативной памяти (shm_open), отображение (mmap) выполняется на дескриптор, который возвращает shm_open, а не на файл в файловой системе. Имена общей памяти и семафоров не должны быть константами, чтобы разные экземпляры пары программ server и client не пересекались. Использование старого API (shmget, shmat и т.д.) запрещено.

Условие лабораторной работы №1: родительский процесс создает два дочерних процесса. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла на запись для child1. Аналогично для второй строки и процесса child2. Родительский и дочерний процессы должны быть представлены разными программами.

Правило фильтрации: с вероятностью 80% строки отправляются child1, иначе child2. Дочерние процессы удаляют все гласные из строк и записывают результат в свои файлы.

Ограничения: допускается использование только системных вызовов; из стандартной библиотеки С разрешены только заголовочные файлы stdio.h, threads.h и stdatomic.h.

Общий метод и алгоритм решения

В лабораторной работе использованы следующие системные вызовы и функции POSIX:

- int shm_open(const char *name, int oflag, mode_t mode); – создает или открывает именованный объект общей памяти (находится в оперативной памяти, в /dev/shm) и возвращает файловый дескриптор.
- int shm_unlink(const char *name); – удаляет имя объекта общей памяти из системы.
- int ftruncate(int fd, off_t length); – задает размер объекта общей памяти (новый объект имеет размер 0).
- void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset); – отображает объект в адресное пространство процесса; с флагом MAP_SHARED изменения видны всем процессам, отобразившим этот объект.
- int munmap(void *addr, size_t length); – удаляет отображение из адресного пространства процесса.
- sem_t *sem_open(const char *name, int oflag, ...); – создает или открывает именованный семафор (при создании указываются права и начальное значение).
- int sem_wait(sem_t *sem); – если значение семафора больше нуля, уменьшает его на единицу, иначе блокирует процесс до тех пор, пока значение не станет положительным.
- int sem_post(sem_t *sem); – увеличивает значение семафора на единицу и будит ожидающий процесс.
- int sem_close(sem_t *sem); – закрывает семафор в текущем процессе.
- int sem_unlink(const char *name); – удаляет имя семафора из системы.
- pid_t fork(void); – создает дочерний процесс.

- int execv(const char *path, char *const argv[]); – заменяет образ процесса образом другой программы (client).
- int dup2(int oldfd, int newfd); – делает newfd копией oldfd (используется для перенаправления stdout дочернего процесса в файл).
- int open(const char *path, int flags, mode_t mode); – открывает файл (выходные файлы и /dev/urandom).
- ssize_t read(int fd, void *buf, size_t count); и ssize_t write(int fd, const void *buf, size_t count); – чтение и запись в файловый дескриптор.
- int close(int fd); – закрывает файловый дескриптор.
- pid_t waitpid(pid_t pid, int *status, int options); – ожидает завершения указанного дочернего процесса.
- pid_t getpid(void); – возвращает идентификатор текущего процесса (используется для построения уникальных имен).
- ssize_t readlink(const char *path, char *buf, size_t size); – читает значение символической ссылки (/proc/self/exe, чтобы найти каталог с программами).

Вместо каналов (pipe) из лабораторной работы №1 процессы обмениваются данными через общую память. Родительский процесс создает именованный объект общей памяти, и оба процесса отображают его в свои адресные пространства вызовом mmap с флагом MAP_SHARED. В таблицах страниц обоих процессов виртуальные страницы указывают на один и тот же физический фрейм, поэтому запись одного процесса сразу видна другому, а копирование данных через ядро (как при read/write для pipe) не требуется.

Программа server (родитель) считывает из стандартного ввода первые две строки – имена файлов – и открывает эти файлы на запись. Затем для каждого из двух дочерних процессов создается свой канал связи: объект общей памяти размером 4096 байт и два именованных семафора. После этого server дважды вызывает fork; в потомке стандартный вывод перенаправляется на выходной файл (dup2), и вызывается execv, запускающий программу client. Имена общей памяти и семафоров передаются клиенту через аргументы командной строки.

Имена объектов не являются константами: они формируются с использованием идентификатора процесса-родителя (например, /lab3-shm1-106, /lab3-ready1-106, /lab3-empty1-106). Благодаря этому одновременно запущенные экземпляры пары программ не мешают друг другу. Объекты создаются с флагом O_EXCL, поэтому попытка использовать уже существующее имя приводит к ошибке, а не к порче чужих данных.

Структура общей памяти: первые 4 байта – длина сообщения (uint32_t), далее текст строки. Специальное значение длины UINT32_MAX означает сообщение «конец».

Синхронизация выполняется двумя семафорами на каждый канал по схеме «производитель – потребитель». Семафор empty («буфер свободен») имеет начальное значение 1, семафор ready («данные готовы») – начальное значение 0. Server для отправки строки выполняет sem_wait(empty), записывает длину и текст в общую память и выполняет sem_post(ready). Client выполняет sem_wait(ready), читает сообщение, обрабатывает его и выполняет sem_post(empty). Таким образом, в каждый момент времени с буфером работает только один из процессов, гонки данных нет, а ожидающий процесс блокируется на семафоре и не расходует процессорное время (в отличие от активного ожидания в цикле).

Правило фильтрации реализовано в server: для каждой строки из /dev/urandom читается 4-байтное число r, и если r % 100 < 80, строка отправляется первому клиенту, иначе второму (функция rand() из stdlib.h использовать нельзя). Client копирует из полученной строки все символы, кроме

гласных английского алфавита (a, e, i, o, u в любом регистре), и записывает результат в стандартный вывод, который перенаправлен в файл.

Завершение работы: когда ввод закончился (Ctrl-D), server отправляет каждому клиенту сообщение с длиной UINT32_MAX, ожидает завершения клиентов (waitpid), затем закрывает семафоры (sem_close), снимает отображение (munmap), закрывает дескрипторы и удаляет имена объектов (sem_unlink, shm_unlink), чтобы они не оставались в системе. Client при получении сообщения «конец» выходит из цикла и также закрывает свои ресурсы.

Обработка ошибок: проверяются результаты всех системных вызовов (shm_open, ftruncate, mmap, sem_open, sem_wait, sem_post, open, read, fork, execv). При ошибке выводится сообщение в stderr, функция die() удаляет уже созданные объекты общей памяти и семафоры и завершает программу с ненулевым кодом.

Код программы

server.c

```
#include <stdint.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <semaphore.h>
#include <stdio.h>

#define SHM_SIZE 4096
#define LINE_MAX_LEN 4092

static char CHILD_PROGRAM_NAME[] = "client";

typedef struct {
    char shm_name[64];
    char ready_name[64];
    char empty_name[64];
    int shm;
    char *buf;
    sem_t *ready;
    sem_t *empty;
    pid_t pid;
} channel;

static channel ch[2];
static int created = 0;

static void print_err(const char *msg) {
    size_t n = 0;
    while (msg[n])
       ++n;
    ssize_t w = write(STDERR_FILENO, msg, n);
    (void)w;
}

static void cleanup(void) {
    for (int i = 0; i < created; ++i) {
       sem_unlink(ch[i].ready_name);
       sem_unlink(ch[i].empty_name);
       shm_unlink(ch[i].shm_name);
    }
}

static void die(const char *msg) {
```

```
    print_err(msg);
    cleanup();
    _exit(1);
}

static int read_line(char *buf, int max) {
    int i = 0;
    char c;
    ssize_t r;
    while (i < max - 1 && (r = read(STDIN_FILENO, &c, 1)) > 0) {
       buf[i++] = c;
       if (c == '\n')
          break;
    }
    if (i == 0 && r < 0)
       die("error: failed to read from stdin\n");
    buf[i] = '\0';
    return i;
}

static void wait_sem(sem_t *sem) {
    if (sem_wait(sem) == -1)
       die("error: sem_wait failed\n");
}

static void post_sem(sem_t *sem) {
    if (sem_post(sem) == -1)
       die("error: sem_post failed\n");
}

static void create_channel(channel *c, int index, int parent_pid) {
    snprintf(c->shm_name, sizeof(c->shm_name), "/lab3-shm%d-%d", index, parent_pid);
    snprintf(c->ready_name, sizeof(c->ready_name), "/lab3-ready%d-%d", index,
parent_pid);
    snprintf(c->empty_name, sizeof(c->empty_name), "/lab3-empty%d-%d", index,
parent_pid);
    created = index + 1;
    c->shm = shm_open(c->shm_name, O_RDWR | O_CREAT | O_EXCL, 0600);
    if (c->shm == -1)
       die("error: failed to create SHM\n");
    if (ftruncate(c->shm, SHM_SIZE) == -1)
       die("error: failed to resize SHM\n");
    c->buf = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, c->shm, 0);
    if (c->buf == MAP_FAILED)
       die("error: failed to map SHM\n");
    c->ready = sem_open(c->ready_name, O_RDWR | O_CREAT | O_EXCL, 0600, 0);
    if (c->ready == SEM_FAILED)
       die("error: failed to create semaphore\n");
    c->empty = sem_open(c->empty_name, O_RDWR | O_CREAT | O_EXCL, 0600, 1);
    if (c->empty == SEM_FAILED)
       die("error: failed to create semaphore\n");
}

static void send_line(channel *c, const char *data, uint32_t len) {
    wait_sem(c->empty);
    uint32_t *length = (uint32_t *)c->buf;
    char *text = c->buf + sizeof(uint32_t);
    for (uint32_t i = 0; i < len; ++i)
       text[i] = data[i];
    *length = len;
    post_sem(c->ready);
}

static void send_stop(channel *c) {
    wait_sem(c->empty);
```

```
    uint32_t *length = (uint32_t *)c->buf;
    *length = UINT32_MAX;
    post_sem(c->ready);
}

static void finish(int started) {
    for (int i = 0; i < started; ++i)
       send_stop(&ch[i]);
    for (int i = 0; i < started; ++i)
       waitpid(ch[i].pid, NULL, 0);
    for (int i = 0; i < created; ++i) {
       sem_close(ch[i].ready);
       sem_close(ch[i].empty);
       munmap(ch[i].buf, SHM_SIZE);
       close(ch[i].shm);
    }
    cleanup();
}

int main() {
    char progpath[1024]; {
       ssize_t len = readlink("/proc/self/exe", progpath, sizeof(progpath) - 1);
       if (len == -1)
          die("error: failed to read full program path\n");
       while (progpath[len] != '/')
          --len;
       progpath[len] = '\0';
    }
    char name1[256];
    char name2[256];
    int len1 = read_line(name1, sizeof(name1));
    int len2 = read_line(name2, sizeof(name2));
    if (len1 <= 1 || len2 <= 1)
       die("error: need two file names\n");
    name1[len1 - 1] = '\0';
    name2[len2 - 1] = '\0';
    int files[2];
    files[0] = open(name1, O_WRONLY | O_CREAT | O_TRUNC, 0600);
    files[1] = open(name2, O_WRONLY | O_CREAT | O_TRUNC, 0600);
    if (files[0] == -1 || files[1] == -1)
       die("error: failed to open file\n");
    int rnd = open("/dev/urandom", O_RDONLY);
    if (rnd == -1)
       die("error: failed to open /dev/urandom\n");
    int parent_pid = (int)getpid();
    for (int i = 0; i < 2; ++i)
       create_channel(&ch[i], i + 1, parent_pid);
    int started = 0;
    for (int i = 0; i < 2; ++i) {
       const pid_t child = fork();
       if (child == -1) {
          print_err("error: failed to spawn new process\n");
          finish(started);
          return 1;
       }
       if (child == 0) {
          dup2(files[i], STDOUT_FILENO);
          close(files[0]);
          close(files[1]);
          close(rnd);
          close(ch[0].shm);
          close(ch[1].shm);
          char path[1100];
          snprintf(path, sizeof(path) - 1, "%s/%s", progpath, CHILD_PROGRAM_NAME);
          char *const args[] = {CHILD_PROGRAM_NAME, ch[i].shm_name, ch[i].ready_name,
```

```
ch[i].empty_name, NULL};
          execv(path, args);
          print_err("error: failed to exec into new exectuable image\n");
          _exit(1);
       }
       ch[i].pid = child;
       ++started;
    }
    close(files[0]);
    close(files[1]);
    char buf[LINE_MAX_LEN];
    int bytes;
    while ((bytes = read_line(buf, sizeof(buf))) > 0) {
       uint32_t r = 0;
       if (read(rnd, &r, sizeof(r)) != sizeof(r))
          die("error: failed to read /dev/urandom\n");
       if (r % 100 < 80)
          send_line(&ch[0], buf, bytes);
       else
          send_line(&ch[1], buf, bytes);
    }
    close(rnd);
    finish(started);
    return 0;
}
```

client.c

```
#include <stdint.h>
#include <stdbool.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <semaphore.h>

#define SHM_SIZE 4096

static void print_err(const char *msg) {
    size_t n = 0;
    while (msg[n])
       ++n;
    ssize_t w = write(STDERR_FILENO, msg, n);
    (void)w;
}

static bool is_vowel(char c) {
    return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' ||
           c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U';
}

int main(int argc, char **argv) {
    if (argc != 4) {
       print_err("usage: client shm_name ready_sem empty_sem\n");
       _exit(1);
    }
    int shm = shm_open(argv[1], O_RDWR, 0);
    if (shm == -1) {
       print_err("error: failed to open SHM\n");
       _exit(1);
    }
    char *shm_buf = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm, 0);
    if (shm_buf == MAP_FAILED) {
       print_err("error: failed to map SHM\n");
       _exit(1);
```

```
        }
        sem_t *ready = sem_open(argv[2], O_RDWR);
        sem_t *empty = sem_open(argv[3], O_RDWR);
        if (ready == SEM_FAILED || empty == SEM_FAILED) {
           print_err("error: failed to open semaphore\n");
           _exit(1);
        }
        bool running = true;
        while (running) {
           if (sem_wait(ready) == -1) {
              print_err("error: sem_wait failed\n");
              _exit(1);
           }
           uint32_t *length = (uint32_t *)shm_buf;
           char *text = shm_buf + sizeof(uint32_t);
           if (*length == UINT32_MAX) {
              running = false;
           } else if (*length <= SHM_SIZE - sizeof(uint32_t)) {
              char out[SHM_SIZE];
              uint32_t o = 0;
              for (uint32_t i = 0; i < *length; ++i) {
                 if (!is_vowel(text[i]))
                    out[o++] = text[i];
              }
              if (o > 0 && write(STDOUT_FILENO, out, o) != (ssize_t)o) {
                 print_err("error: failed to write to file\n");
                 _exit(1);
              }
           }
           if (sem_post(empty) == -1) {
              print_err("error: sem_post failed\n");
              _exit(1);
           }
        }
        sem_close(ready);
        sem_close(empty);
        munmap(shm_buf, SHM_SIZE);
        close(shm);
    }
```

Протокол работы программы

Тестирование:

```
$ cc -o client client.c
$ cc -o server server.c
$ ./server
file1.txt
file2.txt
Hello world
Operating systems
process and pipe
linux kernel
fork and exec
string six
lab number one
variant nineteen
parent child
student
^D
$ cat file1.txt
prtng systms
prcss nd pp
lnx krnl
```

```
frk nd xc
strng sx
lb nmbr n
vrnt nntn
prnt chld
stdnt
$ cat file2.txt
Hll wrld
```

Во время работы программы (в другом терминале) созданные объекты общей памяти и семафоров видны в /dev/shm, а процессы client запущены с именами объектов в аргументах. После завершения программы объекты удаляются:

```
$ ls /dev/shm | grep lab3
lab3-shm1-106
lab3-shm2-106
sem.lab3-empty1-106
sem.lab3-empty2-106
sem.lab3-ready1-106
sem.lab3-ready2-106
$ ps -o pid,ppid,cmd --ppid 106
  PID  PPID CMD
  108   106 client /lab3-shm1-106 /lab3-ready1-106 /lab3-empty1-106
  109   106 client /lab3-shm2-106 /lab3-ready2-106 /lab3-empty2-106
(после завершения ./server)
$ ls /dev/shm | grep lab3
(пусто)
```

Проверка распределения на 1000 строк (в файл первого клиента попало 793 строки, второго – 207, что соответствует вероятности 80% / 20%; содержимое файлов сверено с эталонным удалением гласных):

```
$ (printf 'big1.txt\nbig2.txt\n'; cat lines.txt) | ./server
$ wc -l big1.txt big2.txt
  793 big1.txt
  207 big2.txt
$ head -2 big1.txt
Ln nmbr 0 wth sm Txt
Ln nmbr 1 wth sm Txt
```

Вывод

В ходе работы лабораторная работа №1 была переработана: вместо каналов (pipe) обмен данными между процессами server и client выполняется через именованную общую память (shm_open, ftruncate, mmap с MAP_SHARED), а чтение и запись синхронизируются двумя именованными семафорами по схеме «производитель – потребитель». Программа работает корректно: строки распределяются между клиентами в соотношении, близком к 80% / 20% (793 и 207 из 1000), гласные удаляются, а после завершения в /dev/shm не остается созданных объектов.

При выполнении работы важно было различать shared memory (объект в оперативной памяти, создаваемый через shm_open) и memory mapping (механизм mmap, подключающий объект к адресному пространству), а также не делать имена объектов константами (для этого в имя добавляется PID родителя). Для корректной работы пришлось аккуратно удалять объекты через sem_unlink и shm_unlink, иначе они остаются в системе до перезагрузки. Ограничение реализации: если клиент не удалось запустить (execv завершился с ошибкой), server будет ожидать семафор бесконечно.